

04_events-py

#python #api #swagger

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ api/  
            └─ routes/  
                └─ events.py
```

El archivo `events.py` se encuentra dentro de la carpeta de rutas de la API:

```
backend/app/api/routes/
```

Este archivo define los endpoints relacionados con la creación simple y consulta de eventos almacenados en la base de datos.

Las rutas principales son:

```
POST /events  
GET /events
```

A diferencia de `ingest.py`, este archivo no ejecuta la lógica completa de evaluación de reglas ni generación de alertas. Su función principal es trabajar directamente con eventos.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,320p' backend/app/api/routes/events.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,320p'
```

Indica que se impriman las líneas de la 1 a la 320.

```
backend/app/api/routes/events.py
```

Ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```

from typing import Optional

from fastapi import APIRouter, Depends, Query
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.models.event import Event
from app.schemas.event import EventCreate, EventOut

router = APIRouter(prefix="/events", tags=["events"])

@router.post("", response_model=EventOut)
def create_event(payload: EventCreate, db: Session = Depends(get_db)):
    ev = Event(source=payload.source, severity=payload.severity, message=payload.message)
    db.add(ev)
    db.commit()
    db.refresh(ev)
    return ev

@router.get("", response_model=list[EventOut])
def list_events(
    limit: int = Query(50, ge=1, le=500),
    before_id: Optional[int] = Query(None, ge=1),
    source: Optional[str] = None,
    severity_min: Optional[int] = Query(None, ge=0, le=10),
    severity_max: Optional[int] = Query(None, ge=0, le=10),
    q: Optional[str] = None,
    meta_key: Optional[str] = None,
    meta_value: Optional[str] = None,
    db: Session = Depends(get_db),
):
    stmt = select(Event)

    if before_id is not None:
        stmt = stmt.where(Event.id < before_id)

    if source:
        stmt = stmt.where(Event.source == source)

    if severity_min is not None:
        stmt = stmt.where(Event.severity >= severity_min)

    if severity_max is not None:
        stmt = stmt.where(Event.severity <= severity_max)

    if q:
        stmt = stmt.where(Event.message.ilike(f"%{q}%"))

    if meta_key and meta_value:
        stmt = stmt.where(Event.meta[meta_key].astext == meta_value)
    elif meta_key:
        stmt = stmt.where(Event.meta.has_key(meta_key)) # noqa: W601

    stmt = stmt.order_by(Event.id.desc()).limit(limit)
    return db.execute(stmt).scalars().all()

```

4 Función general del archivo

El archivo `events.py` define rutas para crear y consultar eventos.

Expone dos endpoints principales:

```
POST /events
GET /events
```

El endpoint `POST /events` permite crear un evento simple a partir de un payload `EventCreate`.

El endpoint `GET /events` permite listar eventos almacenados en PostgreSQL, aplicando filtros opcionales.

Este archivo es útil para:

- Crear eventos simples sin pasar por la lógica completa de `/ingest`.
- Consultar eventos guardados.
- Filtrar eventos por origen, severidad, texto o metadatos.
- Hacer paginación básica usando `before_id`.
- Servir datos al frontend o a pruebas manuales con `curl/Swagger`.

Diferencia importante:

```
POST /events
  ↓
crea evento simple

POST /ingest
  ↓
crea evento + evalúa reglas + puede generar alertas
```

5 Estructura general del archivo

El archivo puede dividirse en seis bloques:

```
from typing import Optional
```

Importación para tipos opcionales.

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy import select
from sqlalchemy.orm import Session
```

Importaciones de FastAPI y SQLAlchemy.

```
from app.db.session import get_db
from app.models.event import Event
from app.schemas.event import EventCreate, EventOut
```

Importaciones internas del proyecto.

```
router = APIRouter(prefix="/events", tags=["events"])
```

Creación del router `/events`.

```
@router.post("", response_model=EventOut)
def create_event(...):
```

Endpoint para crear eventos.

```
@router.get("", response_model=list[EventOut])
def list_events(...):
```

Endpoint para listar y filtrar eventos.

Visualmente:

```

events.py
├─ Importaciones
├─ Router /events
├─ POST /events
│   ├── recibe EventCreate
│   ├── crea Event
│   ├── db.add()
│   ├── db.commit()
│   ├── db.refresh()
│   └─ devuelve EventOut
└─ GET /events
    ├── recibe filtros opcionales
    ├── construye select(Event)
    ├── aplica filtros
    ├── ordena por id descendente
    ├── limita resultados
    └─ devuelve list[EventOut]

```

6 Análisis línea por línea

Importación de `Optional`

```
from typing import Optional
```

Esta línea importa `Optional` desde el módulo estándar `typing`.

`Optional` indica que un valor puede ser del tipo indicado o puede ser `None`.

En este archivo se usa para parámetros de filtrado:

```

before_id: Optional[int]
source: Optional[str]
severity_min: Optional[int]
severity_max: Optional[int]
q: Optional[str]
meta_key: Optional[str]
meta_value: Optional[str]

```

Esto significa que todos esos filtros son opcionales.

El usuario puede llamar a:

```
GET /events
```

sin indicar ningún filtro.

Importación de `FastAPI`

```
from fastapi import APIRouter, Depends, Query
```

Esta línea importa tres elementos desde `FastAPI`.

`APIRouter`

`APIRouter` permite crear un grupo de rutas separado de la aplicación principal.

En este archivo se usa aquí:

```
router = APIRouter(prefix="/events", tags=["events"])
```

Después, este router se incluye en `main.py`:

```
app.include_router(events_router)
```

Depends

`Depends` permite usar dependencias de FastAPI.

En este archivo se utiliza para obtener una sesión de base de datos:

```
db: Session = Depends(get_db)
```

Query

`Query` permite definir parámetros de consulta y aplicar validaciones.

Ejemplo:

```
limit: int = Query(50, ge=1, le=500)
```

Esto define un parámetro `limit` con valor por defecto 50, mínimo 1 y máximo 500.

Importación de `select`

```
from sqlalchemy import select
```

Esta línea importa `select` desde SQLAlchemy.

`select` permite construir consultas SQL usando sintaxis Python.

En este archivo se usa aquí:

```
stmt = select(Event)
```

Esto crea una consulta base para seleccionar eventos.

Conceptualmente equivale a:

```
SELECT * FROM events;
```

Importación de `Session`

```
from sqlalchemy.orm import Session
```

Importa el tipo `Session` de SQLAlchemy.

Se usa como anotación de tipo para la sesión de base de datos:

```
db: Session = Depends(get_db)
```

Esto indica que `db` será una sesión SQLAlchemy.

Importación de `get_db`

```
from app.db.session import get_db
```

Importa la dependencia que proporciona sesiones de base de datos.

Esta función crea una sesión con `SessionLocal()` y la cierra al finalizar la petición.

Se utiliza en los dos endpoints del archivo:

```
db: Session = Depends(get_db)
```

Importación del modelo `Event`

```
from app.models.event import Event
```

Importa el modelo SQLAlchemy `Event`.

Este modelo representa la tabla:

```
events
```

Se utiliza para:

- Crear nuevos eventos.
- Consultar eventos existentes.
- Aplicar filtros sobre columnas de eventos.

Importación de schemas

```
from app.schemas.event import EventCreate, EventOut
```

Importa dos schemas de Pydantic:

```
EventCreate  
EventOut
```

`EventCreate`

Se usa como schema de entrada para crear eventos simples.

Aparece aquí:

```
def create_event(payload: EventCreate, db: Session = Depends(get_db)):
```

`EventOut`

Se usa como schema de salida.

Aparece aquí:

```
@router.post("", response_model=EventOut)
```

y:

```
@router.get("", response_model=list[EventOut])
```

Esto permite que FastAPI devuelva eventos con una estructura controlada.

Creación del router

```
router = APIRouter(prefix="/events", tags=["events"])
```

Esta línea crea el router de eventos.

Desglose:

```
router
```

Variable que contiene el router.

```
APIRouter(...)
```

Crea una instancia de router de FastAPI.

```
prefix="/events"
```

Todas las rutas definidas en este archivo empezarán por `/events` .

```
tags=["events"]
```

Agrupar estas rutas bajo la etiqueta `events` en Swagger.

Como los decoradores usan cadena vacía:

```
@router.post("")
@router.get("")
```

las rutas finales son:

```
POST /events
GET /events
```

Endpoint `POST /events`

```
@router.post("", response_model=EventOut)
```

Este decorador registra un endpoint HTTP de tipo `POST` .

La ruta relativa es:

```
""
```

Como el router tiene prefijo `/events` , la ruta final es:

```
POST /events
```

El parámetro:

```
response_model=EventOut
```

indica que la respuesta debe adaptarse al schema `EventOut` .

Definición de `create_event`

```
def create_event(payload: EventCreate, db: Session = Depends(get_db)):
```

Define la función que se ejecuta al llamar a `POST /events` .

Parámetros:

```
payload → datos validados con EventCreate
db       → sesión de base de datos
```

FastAPI hace automáticamente:

1. Lee el JSON recibido.
2. Lo valida con `EventCreate`.
3. Ejecuta `get_db()`.
4. Entrega payload y db a la función.

Creación del objeto `Event`

```
ev = Event(source=payload.source, severity=payload.severity, message=payload.message)
```

Crea un objeto SQLAlchemy de tipo `Event`.

Relación entre schema y modelo:

```
payload.source    → ev.source
payload.severity  → ev.severity
payload.message   → ev.message
```

Este endpoint no asigna manualmente:

```
ts
created_at
meta
```

Los campos `ts` y `created_at` tienen valor por defecto en la base de datos.

El campo `meta` no se incluye en `EventCreate`, así que no se establece desde este endpoint.

Añadir evento a la sesión

```
db.add(ev)
```

Añade el evento a la sesión SQLAlchemy.

Esto marca el objeto como pendiente de inserción.

Todavía no está confirmado definitivamente en PostgreSQL hasta que se ejecuta:

```
db.commit()
```

Confirmar transacción

```
db.commit()
```

Confirma la transacción.

En este momento, SQLAlchemy envía la inserción a PostgreSQL y el evento queda guardado.

Actualizar el objeto

```
db.refresh(ev)
```

Actualiza el objeto `ev` con los datos reales generados por la base de datos.

Esto es importante porque PostgreSQL puede haber generado valores automáticamente, como:

```
id
ts
```



```
created_at
```

Después de `db.refresh(ev)` , el objeto tiene esos valores cargados.

Devolver evento

```
return ev
```

Devuelve el objeto `Event` .

Como el endpoint tiene:

```
response_model=EventOut
```

FastAPI convierte el objeto SQLAlchemy en una respuesta JSON con la estructura de `EventOut` .

Endpoint GET /events

```
@router.get("", response_model=list[EventOut])
```

Este decorador registra un endpoint HTTP de tipo `GET` .

La ruta final es:

```
GET /events
```

El parámetro:

```
response_model=list[EventOut]
```

indica que la respuesta será una lista de eventos.

Cada elemento de la lista debe seguir el schema `EventOut` .

Definición de list_events

```
def list_events(
```

Inicia la definición de la función que lista eventos.

La función recibe varios parámetros opcionales para filtrar resultados.

Parámetro limit

```
limit: int = Query(50, ge=1, le=500),
```

Define el número máximo de eventos devueltos.

Desglose:

```
limit
```

Nombre del parámetro.

```
: int
```

Debe ser un entero.

```
Query(50, ge=1, le=500)
```

Define valor por defecto y validaciones.

```
50      → valor por defecto
ge=1    → mínimo 1
le=500  → máximo 500
```

Ejemplos:

```
GET /events
GET /events?limit=10
GET /events?limit=500
```

Parámetro `before_id`

```
before_id: Optional[int] = Query(None, ge=1),
```

Permite paginación básica por identificador.

Si se indica `before_id`, el endpoint devuelve eventos con `id` menor que ese valor.

Ejemplo:

```
GET /events?before_id=100
```

Esto devuelve eventos anteriores al id 100.

Desglose:

```
Optional[int]
```

Puede ser entero o `None`.

```
Query(None, ge=1)
```

Por defecto es `None`, pero si se indica debe ser mayor o igual que 1.

Parámetro `source`

```
source: Optional[str] = None,
```

Filtro opcional por origen del evento.

Ejemplo:

```
GET /events?source=auth
```

Solo devolverá eventos cuyo `source` sea `auth`.

Parámetro `severity_min`

```
severity_min: Optional[int] = Query(None, ge=0, le=10),
```

Filtro opcional por severidad mínima.

Ejemplo:

```
GET /events?severity_min=5
```

Solo devuelve eventos con severidad mayor o igual que 5.

Validaciones:

```
mínimo → 0  
máximo → 10
```

Parámetro `severity_max`

```
severity_max: Optional[int] = Query(None, ge=0, le=10),
```

Filtro opcional por severidad máxima.

Ejemplo:

```
GET /events?severity_max=3
```

Solo devuelve eventos con severidad menor o igual que 3.

Parámetro `q`

```
q: Optional[str] = None,
```

Filtro opcional de búsqueda textual sobre el campo `message`.

Ejemplo:

```
GET /events?q=failed
```

Busca eventos cuyo mensaje contenga `failed`.

Parámetro `meta_key`

```
meta_key: Optional[str] = None,
```

Filtro opcional para buscar eventos que tengan una clave concreta dentro del campo `meta`.

Ejemplo:

```
GET /events?meta_key=host
```

Busca eventos cuyo JSON `meta` tenga la clave `host`.

Parámetro `meta_value`

```
meta_value: Optional[str] = None,
```

Filtro opcional para buscar eventos cuyo `meta` tenga una clave con un valor concreto.

Se utiliza junto con `meta_key`.

Ejemplo:

```
GET /events?meta_key=host&meta_value=server-01
```

Busca eventos cuyo `meta.host` sea `server-01`.

Parámetro `db`

```
db: Session = Depends(get_db),
```

Obtiene una sesión de base de datos mediante la dependencia `get_db`.

Esto permite ejecutar la consulta sobre PostgreSQL.

Cierre de parámetros

```
):
```

Cierra la lista de parámetros de la función.

A partir de aquí empieza el bloque de código de `list_events`.

Consulta base

```
stmt = select(Event)
```

Crea una consulta base para seleccionar eventos.

`stmt` viene de `statement`.

Inicialmente no tiene filtros.

Conceptualmente:

```
SELECT * FROM events;
```

Después, según los parámetros recibidos, se irán añadiendo condiciones con `.where(...)`.

Filtro por `before_id`

```
if before_id is not None:
    stmt = stmt.where(Event.id < before_id)
```

Si el usuario ha enviado `before_id`, se filtran eventos con `id` menor.

Ejemplo:

```
before_id = 100
```

Condición:

```
WHERE id < 100
```

Esto sirve para paginación: pedir eventos anteriores al último visto.

Filtro por `source`

```
if source:
    stmt = stmt.where(Event.source == source)
```

Si el usuario indica `source`, se filtra por origen exacto.

Ejemplo:

```
source = "auth"
```

Condición:

```
WHERE source = 'auth'
```

Filtro por severidad mínima

```
if severity_min is not None:
    stmt = stmt.where(Event.severity >= severity_min)
```

Si se indica `severity_min`, se devuelven eventos con severidad igual o superior.

Ejemplo:

```
severity_min = 5
```

Condición:

```
WHERE severity >= 5
```

Filtro por severidad máxima

```
if severity_max is not None:
    stmt = stmt.where(Event.severity <= severity_max)
```

Si se indica `severity_max`, se devuelven eventos con severidad igual o inferior.

Ejemplo:

```
severity_max = 3
```

Condición:

```
WHERE severity <= 3
```

Filtro por texto en mensaje

```
if q:
    stmt = stmt.where(Event.message.ilike(f"%{q}%"))
```

Si se indica `q`, se busca ese texto dentro del mensaje.

`ilike` realiza una búsqueda no sensible a mayúsculas/minúsculas.

Ejemplo:

```
q = "failed"
```

Condición conceptual:

```
WHERE message ILIKE '%failed%'
```

Esto encontraría mensajes como:

```
Failed login attempt
FAILED LOGIN
```

```
authentication failed
```

Filtro por meta_key y meta_value

```
if meta_key and meta_value:
    stmt = stmt.where(Event.meta[meta_key].astext == meta_value)
```

Si se indican tanto `meta_key` como `meta_value`, se filtra por una clave concreta dentro de `meta` y un valor concreto.

Ejemplo:

```
meta_key=host
meta_value=server-01
```

Condición conceptual:

```
WHERE meta->>'host' = 'server-01'
```

Esto permite buscar eventos con metadatos específicos.

Filtro solo por meta_key

```
elif meta_key:
    stmt = stmt.where(Event.meta.has_key(meta_key)) # noqa: W601
```

Si se indica `meta_key` pero no `meta_value`, se buscan eventos cuyo `meta` contenga esa clave.

Ejemplo:

```
meta_key=host
```

Devuelve eventos que tengan clave `host`, independientemente de su valor.

`has_key` es una operación específica útil con JSONB de PostgreSQL.

El comentario:

```
# noqa: W601
```

indica a herramientas de linting que ignoren una advertencia concreta sobre el uso de `has_key`.

Aunque pueda parecer raro por el nombre, aquí se usa porque SQLAlchemy lo proporciona para consultas JSONB.

Orden y límite

```
stmt = stmt.order_by(Event.id.desc()).limit(limit)
```

Ordena los eventos por `id` descendente y aplica el límite.

Desglose:

```
Event.id.desc()
```

Ordena de mayor a menor.

Esto muestra primero los eventos más recientes si el `id` aumenta con cada inserción.

```
.limit(limit)
```

Limita la cantidad de resultados devueltos.

Ejemplo:

```
limit = 50
```

Devuelve como máximo 50 eventos.

Ejecución de la consulta

```
return db.execute(stmt).scalars().all()
```

Ejecuta la consulta y devuelve los resultados.

Desglose:

```
db.execute(stmt)
```

Ejecuta la consulta SQLAlchemy.

```
.scalars()
```

Extrae los objetos `Event` directamente.

```
.all()
```

Devuelve todos los resultados como lista.

Como el endpoint tiene:

```
response_model=list[EventOut]
```

FastAPI convierte la lista de objetos `Event` en una lista JSON de `EventOut`.

Resultado final del archivo

Este archivo expone dos endpoints:

```
POST /events  
GET /events
```

POST /events :

1. Recibe `EventCreate`.
2. Crea un `Event`.
3. Lo guarda en PostgreSQL.
4. Refresca el objeto.
5. Devuelve `EventOut`.

GET /events :

1. Crea una consulta `select(Event)`.
 2. Aplica filtros opcionales.
 3. Ordena por id descendente.
 4. Limita resultados.
 5. Devuelve `list[EventOut]`.
-

7 Relación con el flujo técnico del laboratorio

`events.py` actúa como módulo de consulta y creación simple de eventos.

Su relación con el flujo del laboratorio es:

```
POST /events
  ↓
EventCreate
  ↓
Event
  ↓
PostgreSQL
  ↓
EventOut
```

Y para consulta:

```
GET /events
  ↓
filtros opcionales
  ↓
select(Event)
  ↓
PostgreSQL
  ↓
lista de EventOut
```

En comparación con `/ingest`:

```
/events → gestión directa de eventos
/ingest → flujo SIEM completo con reglas y alertas
```

Por tanto, `events.py` es útil para consultar el histórico de eventos y validar que la ingesta está funcionando correctamente.

8 Errores típicos o puntos importantes

`POST /events` no genera alertas

Este endpoint solo crea eventos.

No consulta reglas ni crea alertas.

Para probar el flujo completo del SIEM, se debe usar:

```
POST /ingest
```

`POST /events` no acepta meta

El schema `EventCreate` no incluye el campo `meta`.

Por tanto, si se quiere enviar metadatos, conviene usar:

```
POST /ingest
```

`GET /events` permite filtros combinados

Los filtros pueden combinarse.

Ejemplo:

```
GET /events?source=auth&severity_min=3&q=failed
```

Esto devuelve eventos de origen `auth`, severidad mínima 3 y cuyo mensaje contenga `failed`.

before_id sirve para paginación

El filtro:

```
before_id
```

permite cargar eventos anteriores a un determinado identificador.

Esto evita depender de páginas numéricas y funciona bien con orden descendente por `id`.

meta_key depende de JSONB

Los filtros sobre `meta` usan operaciones propias de PostgreSQL/JSONB.

Por eso el proyecto está especialmente alineado con PostgreSQL.

has_key puede marcar advertencias

La línea:

```
Event.meta.has_key(meta_key) # noqa: W601
```

usa una operación que puede provocar advertencia de estilo.

El comentario `# noqa: W601` evita que el linter la marque como problema.

9 Comandos útiles relacionados

Crear evento simple:

```
curl -X POST http://localhost:8000/events \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 3,
  "message": "Simple event created from /events"
}'
```

Listar eventos:

```
curl http://localhost:8000/events
```

Listar 10 eventos:

```
curl "http://localhost:8000/events?limit=10"
```

Filtrar por origen:

```
curl "http://localhost:8000/events?source=auth"
```

Filtrar por severidad mínima:

```
curl "http://localhost:8000/events?severity_min=4"
```

Filtrar por rango de severidad:

```
curl "http://localhost:8000/events?severity_min=2&severity_max=5"
```

Buscar texto en mensaje:

```
curl "http://localhost:8000/events?q=failed"
```

Filtrar por clave `meta` :

```
curl "http://localhost:8000/events?meta_key=host"
```

Filtrar por clave y valor de `meta` :

```
curl "http://localhost:8000/events?meta_key=host&meta_value=server-01"
```

Paginar con `before_id` :

```
curl "http://localhost:8000/events?before_id=100&limit=20"
```

Consultar eventos directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, source, severity, message, meta, created_at  
FROM events ORDER BY id DESC LIMIT 10;"
```

Probar importación del router:

```
docker exec -it siem-api python -c "from app.api.routes.events import router; print(router)"
```

Ver Swagger:

```
http://localhost:8000/docs
```

Con esto queda cerrado el módulo:

```
```text  
04_API-de-ingesta
└─ 05_Analisis-tecnico-de-ingesta
 ├── 01_schema-ingest-py
 ├── 02_schema-event-py
 ├── 03_ingest-py
 └─ 04_events-py
```